



UNIVERSIDADE FEDERAL DO TOCANTINS

Campus de Palmas

Bacharelado em Ciência da Computação

Eliézer Alencar Moreira

Pedro Lucas Moreira Pinto

Pedro Ryan Oliveira de Almeida

Análise empírica de algoritmos de ordenação

Relatório apresentado à disciplina Projeto e Análise de Algoritmos, sob responsabilidade de Prof. Warley Gramacho da Silva, como parte do trabalho sobre análise empírica de algoritmos de ordenação.

Palmas – TO

2026

RESUMO

Este trabalho compara seis implementações de ordenação em Python: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort e Heap Sort. A base experimental contém 54 registros, correspondentes a três tamanhos de entrada (100, 1.000 e 10.000 elementos), três configurações de ordem e seis algoritmos. São examinados o tempo de execução, as comparações e os contadores de trocas ou movimentações. Na entrada aleatória de 10.000 elementos, o Quick Sort registrou o menor tempo, 0,008097 s; na inversamente ordenada, o Merge Sort obteve 0,009501 s; na ordenada, o Bubble Sort com parada antecipada obteve 0,000451 s. A discussão relaciona esses resultados às características das implementações e às complexidades assintóticas. A ausência de repetições documentadas e de entradas parcialmente ordenadas, assim como uma subcontagem de comparações do Quick Sort, limita a generalização das conclusões.

Palavras-chave: ordenação; análise empírica; complexidade; desempenho.

SUMÁRIO

1	INTRODUÇÃO	1
1.1	Fundamentação teórica	1
2	METODOLOGIA	2
2.1	Base experimental e abrangência	2
2.2	Geração das entradas e protocolo	2
2.3	Definição e limitações das métricas	3
3	IMPLEMENTAÇÃO	4
3.1	Bubble, Selection e Insertion Sort	4
3.2	Merge Sort	4
3.3	Quick Sort	4
3.4	Heap Sort	5
4	RESULTADOS	6
4.1	Tabelas das medições	6
4.2	Tempo em função do tamanho	9
4.3	Comparações e movimentações	11
5	ANÁLISE E DISCUSSÃO	15
5.1	Qual algoritmo apresentou melhor desempenho para entradas aleatórias?	15
5.2	Qual apresentou pior desempenho?	15
5.3	O comportamento está de acordo com a complexidade teórica?	15
5.4	Como o tipo de entrada influencia o desempenho?	16
5.5	Quais algoritmos são mais sensíveis à entrada ordenada ou inversa?	16
5.6	Quando um algoritmo quadrático pode ser competitivo?	16

5.7	O comportamento do Quick Sort está de acordo com o esperado? . . .	17
5.8	Quais fatores explicam diferenças entre teoria e experimento?	17
5.9	Qual algoritmo seria indicado para cada cenário?	18
6	CONCLUSÃO	19
	REFERÊNCIAS	20

1 INTRODUÇÃO

Ordenar consiste em reorganizar elementos segundo uma relação de ordem. Além de constituir um problema fundamental da computação, a ordenação é uma etapa de operações como busca, agrupamento e apresentação de dados. Diferentes métodos podem produzir a mesma sequência final com custos bastante distintos, dependendo do tamanho da entrada, de sua organização inicial e das decisões de implementação.

O objetivo deste trabalho é investigar empiricamente essa diferença e relacionar as medições com as complexidades estudadas na disciplina. Foram selecionados os seis algoritmos propostos no enunciado. Bubble Sort permite observar trocas adjacentes e o efeito de uma parada antecipada; Selection Sort oferece uma referência quadrática pouco sensível à ordem inicial; Insertion Sort representa a construção incremental de um prefixo ordenado. Merge Sort e Quick Sort permitem comparar duas formas de divisão e conquista; Heap Sort explora uma estrutura de seleção baseada em heap. Essa diversidade possibilita confrontar adaptação à entrada, custo de comparações e movimentações.

1.1 Fundamentação teórica

A Tabela 1 resume as ordens de crescimento temporal usuais para essas versões. As referências de Sedgewick e Wayne fundamentam as propriedades de Selection e Insertion [1], Merge [2], Quick [3] e Heap [4]. O melhor caso linear do Bubble decorre, especificamente, da interrupção após uma passagem sem trocas no código analisado.

Tabela 1 – Ordens de crescimento temporal de referência.

Algoritmo	Melhor caso	Caso médio	Pior caso
Bubble Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Quick Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$
Heap Sort	$\Omega(n)$	$\Theta(n \log n)$	$\Theta(n \log n)$

No Quick, o caso médio pressupõe uma distribuição adequada das entradas. Para Heap, o melhor caso depende da hipótese sobre chaves: todas iguais podem levar a tempo linear nesta versão com interrupção do heapify; o limite superior é $O(n \log n)$ em qualquer entrada. A tabela não representa medições.

As ordens assintóticas não determinam tempos absolutos: constantes de implementação e o ambiente também influenciam as medições.

2 METODOLOGIA

2.1 Base experimental e abrangência

A análise utiliza a saída textual fornecida pelo grupo, preservada em `dados/stdout_original.txt`, e os gráficos disponíveis no repositório na revisão 76413e7. A saída foi convertida em CSV convencional, sem modificar as métricas, e em três tabelas. Cada linha identifica algoritmo, cenário, tamanho, comparações, contador de movimentações e tempo.

Foram documentadas as combinações de seis algoritmos, $n \in \{100, 1.000, 10.000\}$ e três cenários: aleatório, ordenado em ordem não decrescente e inversamente ordenado em ordem não crescente. Portanto, há $6 \times 3 \times 3 = 54$ registros. Embora o programa ofereça também 500, 5.000, 50.000 e 100.000 elementos, não há resultados desses tamanhos na base recebida. O recorte descrito é o efetivamente registrado; não foi fornecida a justificativa operacional para a seleção desses tamanhos.

Não há experimento de entrada parcialmente ordenada. Consequentemente, este relatório não atribui resultados a esse cenário e reconhece a cobertura incompleta da proposta experimental. Eventuais expectativas a seu respeito são identificadas como teóricas.

2.2 Geração das entradas e protocolo

Os geradores utilizam `random.Random(42)` e inteiros obtidos por `randint(0, size * 10)`, incluindo os extremos. Repetições de valores são possíveis. Para cada tamanho, os cenários ordenado e inverso são produzidos ordenando a sequência pseudoaleatória gerada com a mesma semente; preservam, assim, o mesmo multiconjunto de valores.

O orquestrador gera a entrada-base e passa uma cópia a cada algoritmo. O parâmetro `in_place=True` faz a ordenação ocorrer nessa cópia. A geração da entrada e a cópia feita pelo orquestrador ficam fora da região cronometrada. Os algoritmos são executados em ordem fixa: Bubble, Selection, Insertion, Merge, Quick e Heap. Os cenários seguem a sequência aleatório, ordenado e inverso. Essa ordem não elimina efeitos de aquecimento ou interferência do ambiente.

O tempo é obtido pela diferença entre duas chamadas a `time.perf_counter()`,

relógio destinado à medição de intervalos [5]. A duração inclui as atualizações dos contadores e as chamadas à função de chave realizadas dentro da ordenação. A saída apresenta seis casas decimais; isso é precisão de apresentação, não garantia de exatidão experimental.

O código não contém laço de repetições e há um único registro por combinação. Os valores são tratados como medições individuais, sem média, desvio-padrão ou intervalo de confiança. Os experimentos foram executados, conforme informações fornecidas pelo grupo, em um computador com processador Intel Core i9-14900F, 64 GB de memória RAM, sistema operacional CachyOS (Linux) e Python 3.13. A versão de correção do Python (3.13.x) não foi especificada. Esses dados se referem à máquina dos experimentos originais.

2.3 Definição e limitações das métricas

O tamanho é o número de elementos; o tipo de entrada descreve sua organização inicial. O contador de comparações procura registrar avaliações entre chaves, não testes de limites de laços. Entretanto, no Quick Sort, a escolha da mediana de três incrementa o contador em dois, mesmo quando as expressões condicionais executam mais comparações. Uma verificação com a entrada $[1, 3, 2]$ identificou oito comparações efetivas entre valores e apenas quatro registradas na ordenação completa. Os valores originais são preservados nas tabelas, com essa ressalva, e não servem para quantificar exatamente o trabalho comparativo do Quick.

O campo **swaps**, aqui denominado contador M , também não tem unidade uniforme. Bubble, Selection e Heap contam trocas de pares; Quick inclui trocas executadas mesmo quando os índices coincidem. Insertion conta deslocamentos para a direita, mas não a escrita final do elemento inserido. Merge conta escritas durante a intercalação no vetor principal, sem incluir as cópias feitas por fatiamento para vetores auxiliares. Portanto, M é útil para analisar a evolução de uma implementação, mas não representa um total homogêneo de escritas entre todos os algoritmos. Zero em M no Insertion não significa ausência de atribuições.

3 IMPLEMENTAÇÃO

Os algoritmos ficam em módulos independentes de `src/`. Cada classe fornece o método `sort`, que retorna a lista ordenada e um dicionário de métricas. A função `build_algorithms`, em `src/main.py`, instancia as seis classes. O benchmark reside em `bench/main.py`; `bench/plot.py` produz os gráficos com Matplotlib.

3.1 Bubble, Selection e Insertion Sort

Bubble compara vizinhos e troca pares fora de ordem. Ao final de uma passagem sem trocas, encerra a execução. Em uma entrada ordenada de tamanho $n > 0$, realiza apenas $n - 1$ comparações. Selection percorre o sufixo ainda não ordenado à procura do mínimo e só troca quando o índice escolhido difere da posição corrente. Assim, realiza exatamente $n(n - 1)/2$ comparações, independentemente da ordem inicial.

Insertion armazena o próximo elemento, desloca os anteriores estritamente maiores e insere o elemento na posição encontrada. Entradas com poucas inversões exigem menos deslocamentos. Bubble e Insertion preservam a ordem relativa de chaves iguais nessas versões; Selection pode modificá-la. Essa propriedade é inferida do código, não de uma medição de estabilidade.

3.2 Merge Sort

Merge divide recursivamente o intervalo ao meio e intercala os resultados. A intercalação utiliza listas auxiliares criadas por fatiamento. Ao comparar valores iguais, escolhe o da metade esquerda, preservando estabilidade. O tempo é $\Theta(n \log n)$ e o espaço auxiliar máximo é $O(n)$. O uso de `in_place=True` significa que o resultado substitui o conteúdo da lista recebida, não que o algoritmo dispensa memória auxiliar.

3.3 Quick Sort

Quick emprega uma pilha explícita de intervalos. Seleciona o pivô pela mediana dos elementos inicial, central e final e o move para a extremidade direita. A partição segue o esquema de Lomuto, usando comparação menor ou igual ao pivô. O intervalo maior é empilhado primeiro, de modo que o menor seja processado antes pela pilha LIFO.

Essa escolha de pivô difere de versões que usam sempre o primeiro ou o último

elemento. Entradas ordenadas não implicam automaticamente seu pior caso. A mediana de três, contudo, não garante partições equilibradas em todas as entradas, e chaves todas iguais podem produzir partições degeneradas nessa implementação. Permanece possível tempo quadrático. A falha de instrumentação descrita na Seção 2.3 não deve ser confundida com uma medição correta do total de comparações.

3.4 Heap Sort

Heap constrói um heap máximo de baixo para cima. Em seguida, troca a raiz com o último elemento da região ativa, reduz essa região e restaura o heap. O procedimento `heapify` é recursivo, de modo que esta versão utiliza até $O(\log n)$ de pilha de chamadas, embora não crie um vetor auxiliar de tamanho n . A construção custa $O(n)$ e o processo completo possui limite superior $O(n \log n)$.

4 RESULTADOS

4.1 Tabelas das medições

As Tabelas 2, 3 e 4 reproduzem os 54 registros recebidos. Não houve correção retroativa de contadores nem repetição dos experimentos para produzir este relatório. A indicação de tempos pequenos deve ser interpretada considerando a resolução impressa e a ausência de repetições.

Tabela 2 – Resultados completos para entrada aleatória. Tempos em segundos.

Algoritmo	n	Comparações	Contador M	Tempo (s)
Bubble Sort	100	4 740	2 298	0,001349
Selection Sort	100	4 950	93	0,000924
Insertion Sort	100	2 394	2 298	0,000590
Merge Sort	100	533	672	0,000382
Quick Sort	100	650	424	0,000277
Heap Sort	100	1 038	594	0,000449
Bubble Sort	1 000	499 290	237 493	0,039372
Selection Sort	1 000	499 500	992	0,019515
Insertion Sort	1 000	238 487	237 493	0,013578
Merge Sort	1 000	8 707	9 976	0,000901
Quick Sort	1 000	10 446	5 531	0,000603
Heap Sort	1 000	16 850	9 099	0,001305
Bubble Sort	10 000	49 988 559	25 076 686	3,227546
Selection Sort	10 000	49 995 000	9 994	1,916343
Insertion Sort	10 000	25 086 677	25 076 686	1,331869
Merge Sort	10 000	120 341	133 616	0,012089
Quick Sort	10 000	143 220	76 376	0,008097
Heap Sort	10 000	235 369	124 246	0,018882

Fonte: saída fornecida pelo grupo. M segue as convenções da Seção 2.3; as comparações do Quick Sort estão subcontadas.

Tabela 3 – Resultados completos para entrada ordenada. Tempos em segundos.

Algoritmo	n	Comparações	Contador M	Tempo (s)
Bubble Sort	100	99	0	0,000004
Selection Sort	100	4 950	0	0,000165
Insertion Sort	100	99	0	0,000005
Merge Sort	100	356	672	0,000056
Quick Sort	100	604	349	0,000041
Heap Sort	100	1 080	638	0,000084
Bubble Sort	1 000	999	0	0,000043
Selection Sort	1 000	499 500	0	0,018787
Insertion Sort	1 000	999	0	0,000060
Merge Sort	1 000	5 044	9 976	0,000718
Quick Sort	1 000	9 060	5 005	0,000485
Heap Sort	1 000	17 575	9 673	0,001396
Bubble Sort	10 000	9 999	0	0,000451
Selection Sort	10 000	49 995 000	0	1,918636
Insertion Sort	10 000	9 999	0	0,000636
Merge Sort	10 000	69 008	133 616	0,009543
Quick Sort	10 000	125 525	66 826	0,006605
Heap Sort	10 000	244 479	131 398	0,019415

Fonte: saída fornecida pelo grupo. M segue as convenções da Seção 2.3; as comparações do Quick Sort estão subcontadas.

Tabela 4 – Resultados completos para entrada inversamente ordenada. Tempos em segundos.

Algoritmo	n	Comparações	Contador M	Tempo (s)
Bubble Sort	100	4 950	4 942	0,000299
Selection Sort	100	4 950	54	0,000173
Insertion Sort	100	4 949	4 942	0,000231
Merge Sort	100	320	672	0,000055
Quick Sort	100	807	509	0,000049
Heap Sort	100	944	515	0,000072
Bubble Sort	1 000	499 500	499 458	0,035396
Selection Sort	1 000	499 500	516	0,019358
Insertion Sort	1 000	499 497	499 458	0,024934
Merge Sort	1 000	4 952	9 976	0,000718
Quick Sort	1 000	14 295	8 928	0,000720
Heap Sort	1 000	15 965	8 321	0,001233
Bubble Sort	10 000	49 995 000	49 994 483	3,918775
Selection Sort	10 000	49 995 000	5 232	1,968484
Insertion Sort	10 000	49 994 980	49 994 483	2,650203
Merge Sort	10 000	64 909	133 616	0,009501
Quick Sort	10 000	230 135	141 224	0,011220
Heap Sort	10 000	226 745	116 721	0,017544

Fonte: saída fornecida pelo grupo. M segue as convenções da Seção 2.3; as comparações do Quick Sort estão subcontadas.

4.2 Tempo em função do tamanho

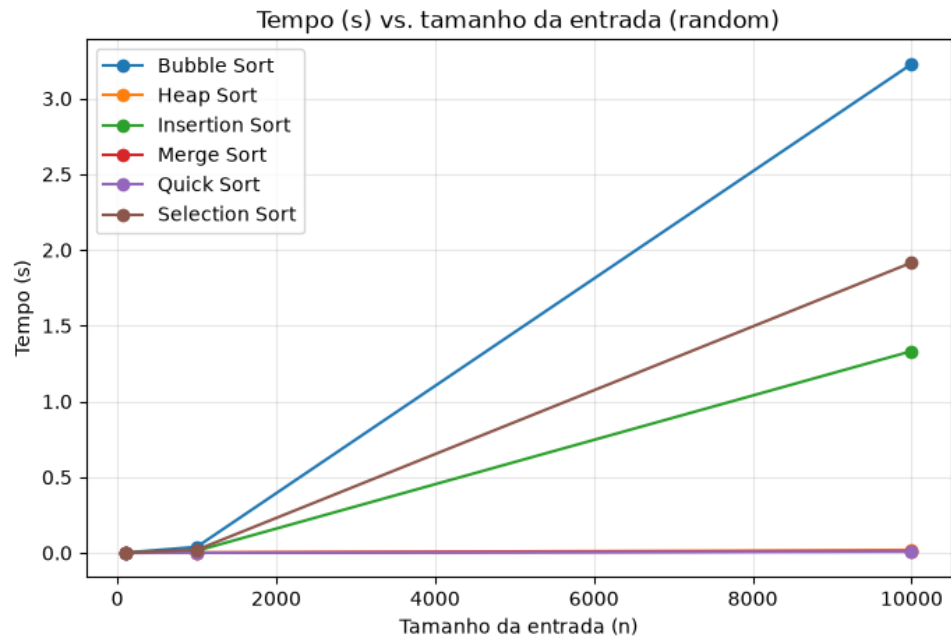


Figura 1 – Tempo para entradas aleatórias.

Fonte: gráficos fornecidos pelo grupo.

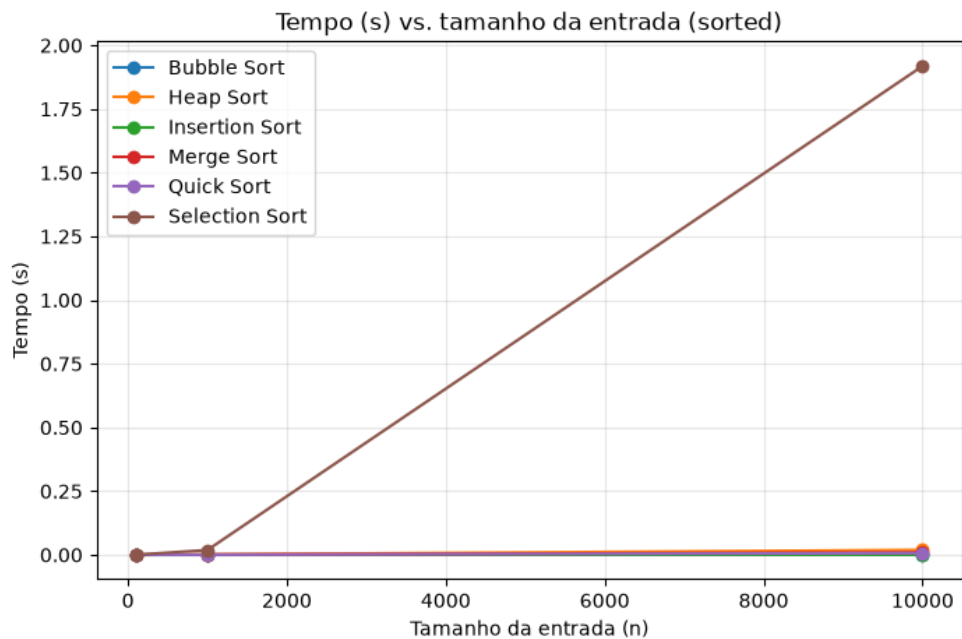


Figura 2 – Tempo para entradas ordenadas.

Fonte: gráficos fornecidos pelo grupo.

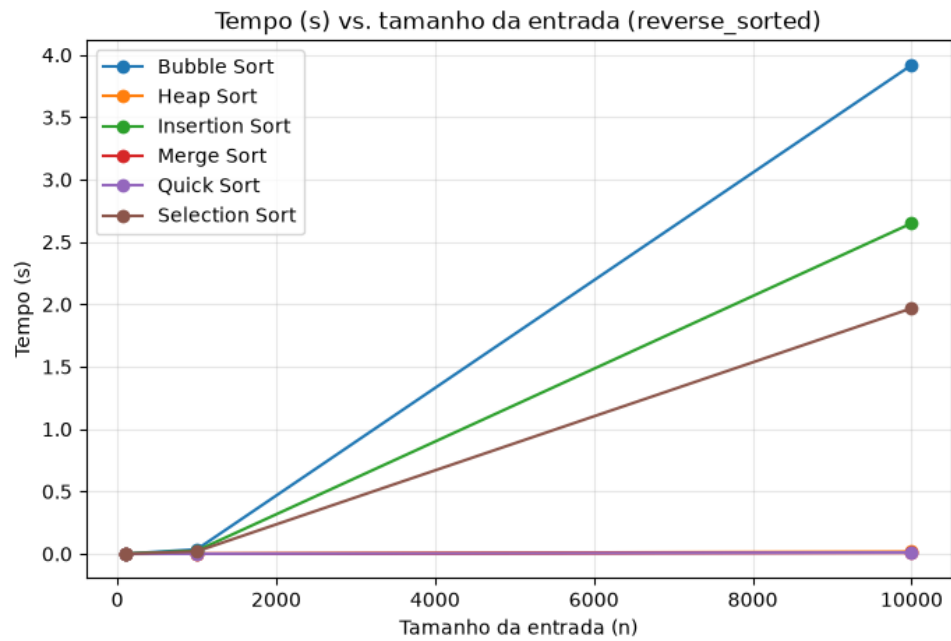


Figura 3 – Tempo para entradas inversamente ordenadas.

Fonte: gráficos fornecidos pelo grupo.

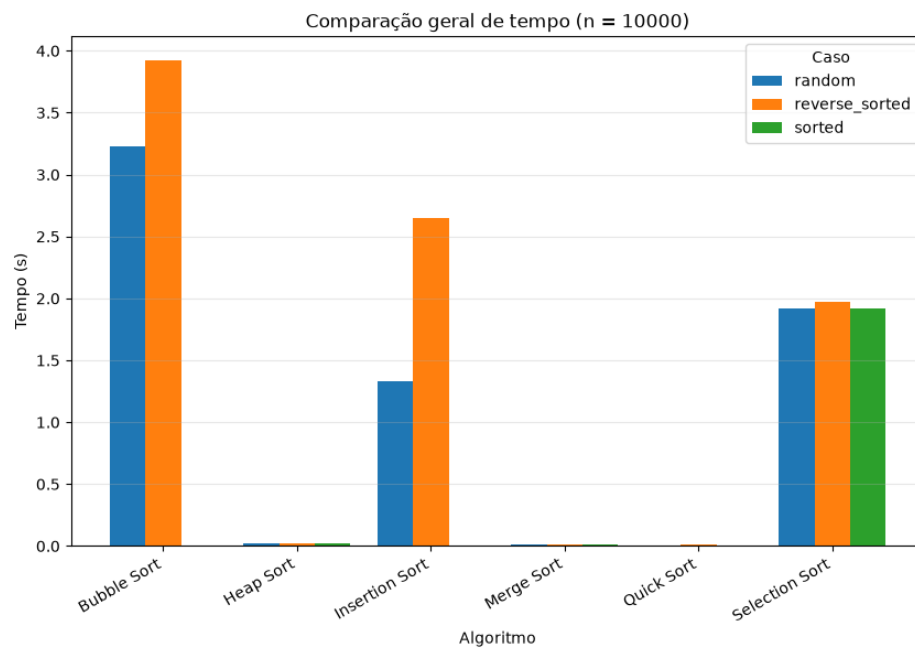


Figura 4 – Comparação temporal dos três cenários em $n = 10.000$.

Fonte: gráficos fornecidos pelo grupo.

As figuras usam escala linear. Com isso, os maiores tempos dos algoritmos quadráticos comprimem visualmente as curvas mais rápidas. A identificação de diferenças pequenas deve recorrer às tabelas. Os segmentos unem somente os tamanhos observados; não são medições intermediárias nem ajustes de uma função de crescimento.

4.3 Comparações e movimentações

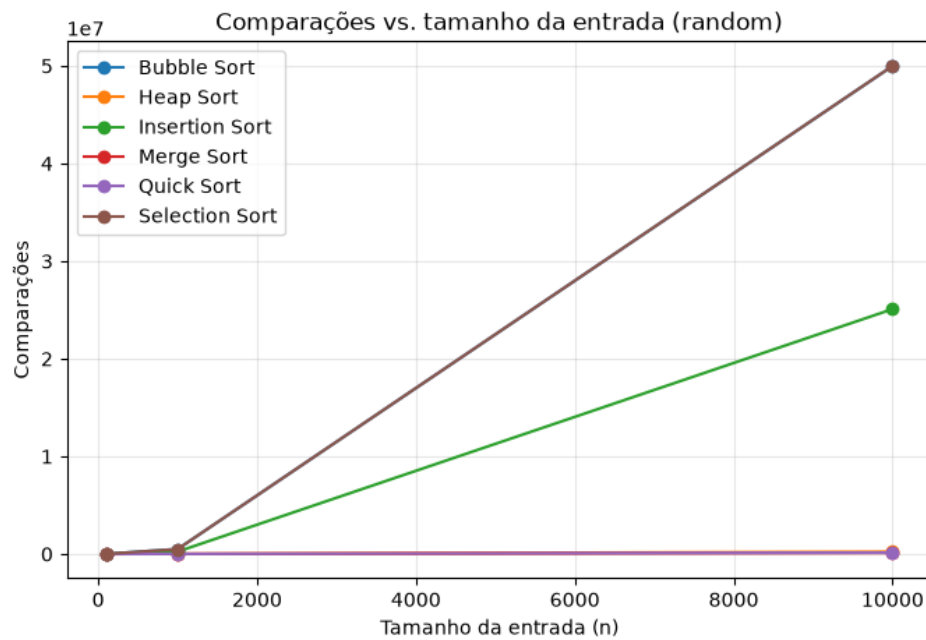


Figura 5 – Comparações registradas em entradas aleatórias. Quick apresenta subcontagem.

Fonte: gráficos fornecidos pelo grupo.

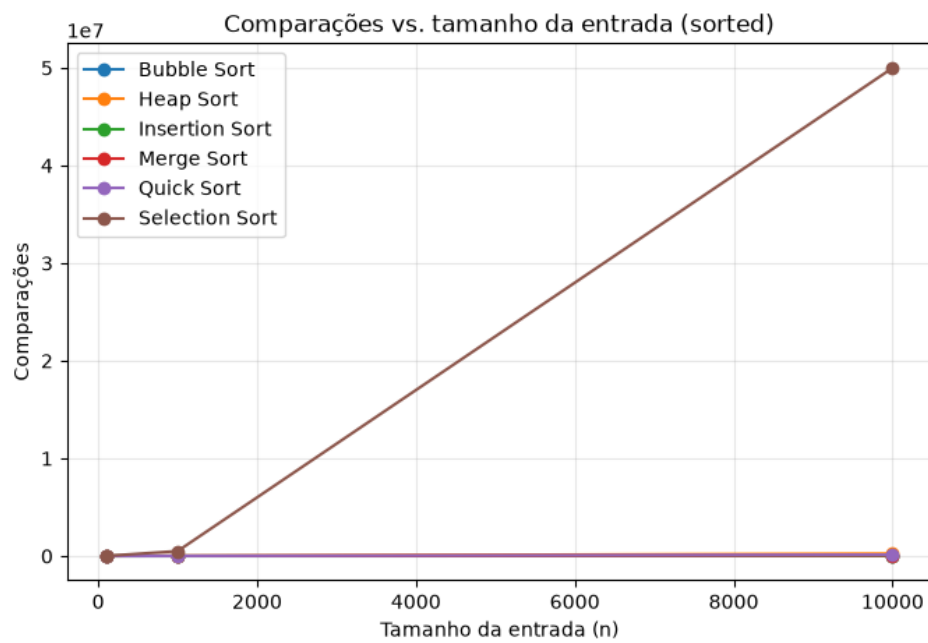


Figura 6 – Comparações registradas em entradas ordenadas. Quick apresenta subcontagem.

Fonte: gráficos fornecidos pelo grupo.

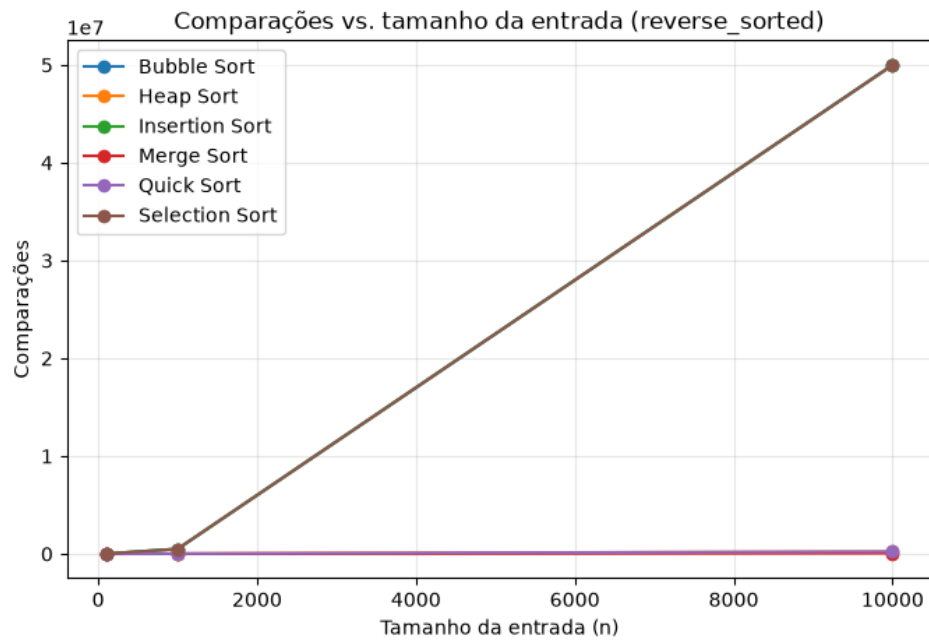


Figura 7 – Comparações registradas em entradas inversas. Quick apresenta subcontagem.

Fonte: gráficos fornecidos pelo grupo.

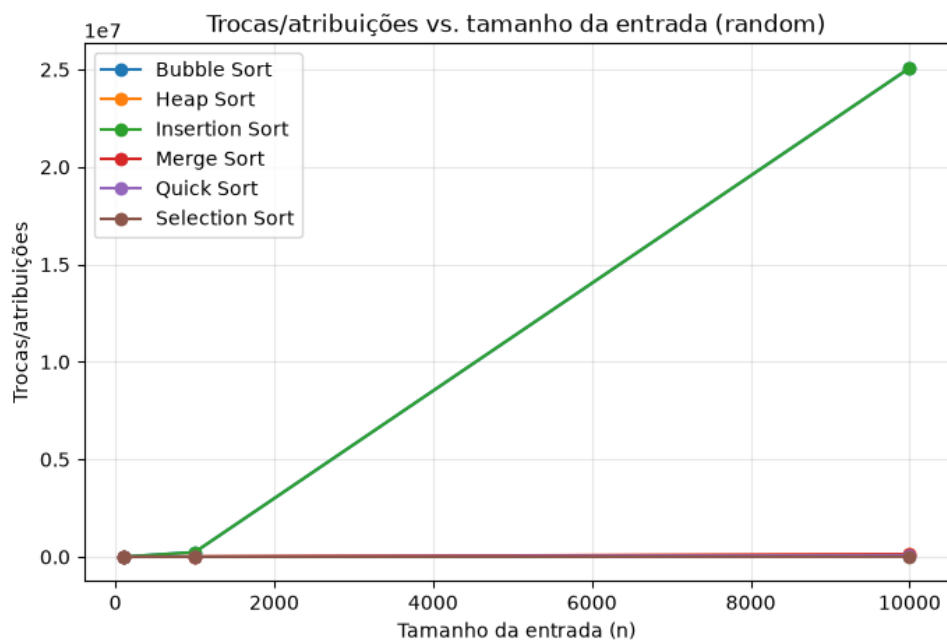


Figura 8 – Contador M em entradas aleatórias; unidades dependem do algoritmo.

Fonte: gráficos fornecidos pelo grupo.

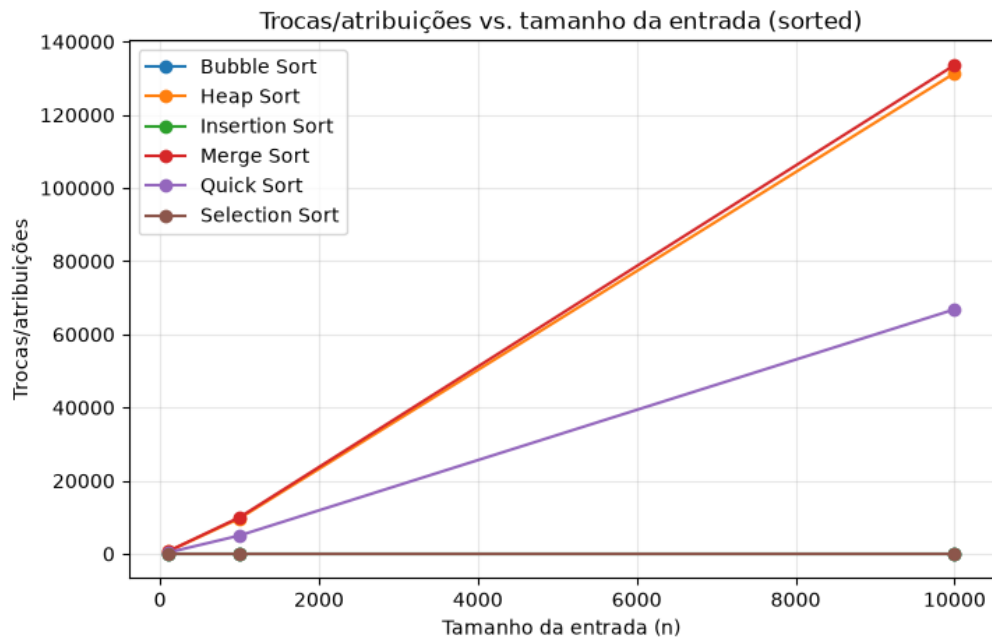


Figura 9 – Contador M em entradas ordenadas; unidades dependem do algoritmo.

Fonte: gráficos fornecidos pelo grupo.

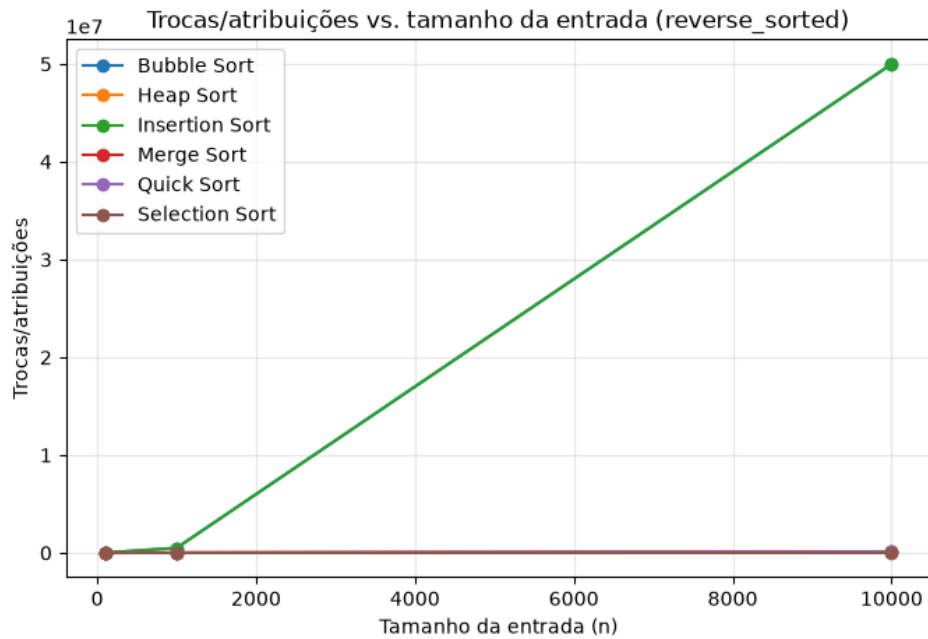


Figura 10 – Contador M em entradas inversas; unidades dependem do algoritmo.

Fonte: gráficos fornecidos pelo grupo.

5 ANÁLISE E DISCUSSÃO

5.1 Qual algoritmo apresentou melhor desempenho para entradas aleatórias?

Quick registrou o menor tempo nos três tamanhos aleatórios: 0,000277 s, 0,000603 s e 0,008097 s. Em 10.000 elementos, Merge levou 0,012089 s e Heap, 0,018882 s. Assim, Quick foi aproximadamente 1,49 vez mais rápido que Merge nessa medição, considerando a razão entre os tempos. Isso caracteriza a execução observada, não superioridade estatisticamente demonstrada ou universal.

5.2 Qual apresentou pior desempenho?

Bubble foi o mais lento nos três tamanhos aleatórios e nos três inversos. Em 10.000 elementos, seus tempos foram 3,227546 s no cenário aleatório e 3,918775 s no inverso. No cenário ordenado, Selection foi o mais lento nos três tamanhos; em 10.000 elementos, levou 1,918636 s. Portanto, o pior desempenho depende do cenário, mesmo dentro desse conjunto de implementações.

5.3 O comportamento está de acordo com a complexidade teórica?

Há evidências compatíveis com a teoria, sobretudo nas contagens. Selection realizou 4.950, 499.500 e 49.995.000 comparações nos três tamanhos, exatamente $n(n-1)/2$, em todos os cenários. Bubble e Insertion, nas entradas ordenadas, registraram 99, 999 e 9.999 comparações, correspondentes a $n-1$.

No cenário aleatório, de 1.000 para 10.000 elementos, o tempo de Selection aumentou aproximadamente 98,19 vezes e o de Insertion, 98,09 vezes, próximos do fator 100 esperado para crescimento quadrático nesse intervalo. Para Merge, o fator foi aproximadamente 13,42, próximo de $10 \log_2(10.000)/\log_2(1.000) \approx 13,33$. Essas razões são descritivas: uma amostra por tamanho não demonstra a ordem assintótica. O fator do Bubble foi aproximadamente 81,98, ilustrando que os tempos não seguem exatamente uma curva teórica.

O contador de escritas de Merge foi 672, 9.976 e 133.616 em cada cenário. Sua igualdade entre ordens iniciais é coerente com a intercalação de todos os elementos em cada

nível. As contagens inválidas do Quick não são usadas para validar quantitativamente sua complexidade.

5.4 Como o tipo de entrada influencia o desempenho?

A ordem inicial altera o trabalho necessário para localizar e reposicionar elementos. Em 10.000 elementos, Insertion passou de 1,331869 s na entrada aleatória para 0,000636 s na ordenada e 2,650203 s na inversa. Bubble passou, respectivamente, de 3,227546 s para 0,000451 s e 3,918775 s. Selection permaneceu entre 1,916343 s e 1,968484 s, pois sua busca de mínimos continua percorrendo os mesmos sufixos.

Merge manteve tempos entre 0,009501 s e 0,012089 s nesse tamanho. A ordem modifica as comparações durante a intercalação, mas não elimina os níveis de divisão. Heap também manteve a mesma ordem de grandeza temporal. Quick apresentou variação associada às partições e ao trabalho de movimentação.

5.5 Quais algoritmos são mais sensíveis à entrada ordenada ou inversa?

Bubble e Insertion foram os mais sensíveis no contraste observado. Na entrada ordenada de 10.000 elementos, ambos fizeram 9.999 comparações e registraram zero no contador M . Na inversa, Bubble fez 49.995.000 comparações e Insertion, 49.994.980. Ambos registraram 49.994.483 trocas ou deslocamentos. Esse total fica ligeiramente abaixo de $n(n-1)/2$ porque a geração permite valores repetidos, e pares iguais não constituem inversões estritas.

Selection oferece o contraste: o número de comparações não mudou com a ordem inicial. Quick foi afetado, mas a entrada inversa de 10.000 elementos não produziu, nesse registro, tempos comparáveis aos algoritmos quadráticos.

5.6 Quando um algoritmo quadrático pode ser competitivo?

Uma classificação de pior caso $O(n^2)$ não exige custo quadrático em toda entrada. Bubble com parada antecipada e Insertion foram competitivos em entradas já ordenadas: em 10.000 elementos, ambos superaram temporalmente Merge, Quick e Heap. Trata-se de uma evidência experimental concreta de adaptação à entrada.

Em entradas com poucas inversões, espera-se bom desempenho de Insertion, cujo

trabalho depende desses deslocamentos [1]. Essa expectativa não foi medida aqui. Constantes menores também podem favorecer algoritmos simples em entradas pequenas, mas os resultados aleatórios de $n = 100$ deste estudo não mostram vitória dos métodos quadráticos. Selection pode ser interessante quando reduzir trocas é prioritário; esse custo diferenciado não foi experimentado.

5.7 O comportamento do Quick Sort está de acordo com o esperado?

O desempenho temporal observado é compatível com uma implementação eficiente nas entradas amostradas. Em $n = 10.000$, os tempos foram 0,008097 s na aleatória, 0,006605 s na ordenada e 0,011220 s na inversa. A mediana de três ajuda a explicar por que a entrada ordenada não desencadeou a degradação típica de uma escolha sistemática de pivô extremo [3].

Entretanto, não se pode concluir que essa estratégia elimina o pior caso ou garante vantagem em todo cenário. Merge foi mais rápido na entrada inversa de 10.000 elementos e também em 1.000 elementos, neste último caso por apenas 0,000002 s, diferença insuficiente para uma conclusão robusta sem repetições. Em 100 elementos inversos, Quick foi mais rápido. A falha do contador impede uma conclusão quantitativa exata baseada nas comparações publicadas.

5.8 Quais fatores explicam diferenças entre teoria e experimento?

A análise assintótica omite constantes e termos menores. No programa, chamadas à função de chave, controle de laços Python, recursão, alocação de listas e instrumentação têm custos distintos. Além disso, comparações são contabilizadas por variáveis locais em algumas rotinas e atributos de objetos em outras, o que também pode afetar tempos.

Escalonamento do sistema, frequência do processador, caches, processos concorrentes e estado do interpretador podem introduzir variação. São fatores possíveis, não causas comprovadas nesta execução, pois essas condições de execução não foram monitoradas. A primeira medição aleatória de 100 elementos, por exemplo, não deve ser usada isoladamente para ajustar uma curva de desempenho. A ordem fixa dos testes e a falta de repetição impedem separar efeitos ocasionais de diferenças persistentes.

Os contadores também exigem cautela: unidades heterogêneas de M não medem o mesmo custo, e a contagem do Quick subestima seu trabalho. Tempos de uma versão

instrumentada não equivalem necessariamente aos de uma versão sem contadores.

5.9 Qual algoritmo seria indicado para cada cenário?

Para as entradas aleatórias estudadas, Quick é a escolha apoiada pelos menores tempos registrados. Para entradas já ordenadas, Bubble otimizado foi o mais rápido nos três tamanhos, com Insertion também apresentando comportamento linear. Para a entrada inversa de 10.000 elementos, Merge é a indicação empírica; nos tamanhos menores, as diferenças entre Merge e Quick precisam de repetições antes de sustentar uma recomendação geral.

Para entradas parcialmente ordenadas, Insertion é um candidato teórico, sujeito a validação experimental. Quando estabilidade é requisito, Merge, Insertion e Bubble preservam a ordem de empates nas implementações analisadas. Quando se deseja limite de pior caso $O(n \log n)$ sem vetor auxiliar linear, Heap é uma alternativa, considerando a pilha recursiva desta versão. Esses critérios de estabilidade e memória foram examinados no código e não medidos como métricas experimentais.

6 CONCLUSÃO

A comparação evidencia que a escolha do algoritmo depende tanto do crescimento da entrada quanto de sua organização inicial. As contagens de Selection confirmam seu trabalho quadrático nos três cenários, enquanto Bubble com parada antecipada e Insertion exploram a ordem prévia para reduzir o trabalho a uma passagem linear. Nas entradas aleatórias de 10.000 elementos, os métodos de crescimento típico $n \log n$ apresentaram tempos muito inferiores aos quadráticos.

Quick registrou o melhor tempo aleatório em todos os tamanhos disponíveis; Merge apresentou o menor tempo inverso em 10.000 elementos; Bubble otimizado liderou as entradas ordenadas. Essas conclusões descrevem os registros disponíveis e não estabelecem uma classificação universal dos algoritmos.

O estudo cobre seis algoritmos, três tamanhos e três cenários, mas permanece limitado por medições individuais, falta do cenário parcialmente ordenado e inconsistências na instrumentação. Para consolidar a análise, recomenda-se corrigir a contagem do Quick, padronizar ou separar trocas e escritas, registrar a versão completa do interpretador e as condições de execução, repetir os ensaios e ampliar as sementes aleatórias. Uma futura avaliação de entradas parcialmente ordenadas permitirá testar diretamente a hipótese de vantagem do Insertion quando há poucas inversões. Os dados atuais sustentam a comparação descritiva apresentada, respeitadas essas limitações.

REFERÊNCIAS

- [1] SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms, 4th edition: Elementary Sorts*. Disponível em: <https://algs4.cs.princeton.edu/21elementary/>. Acesso em: 16 set. 2026. 1, 17
- [2] SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms, 4th edition: Mergesort*. Disponível em: <https://algs4.cs.princeton.edu/22mergesort/>. Acesso em: 16 set. 2026. 1
- [3] SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms, 4th edition: Quicksort*. Disponível em: <https://algs4.cs.princeton.edu/23quicksort/>. Acesso em: 16 set. 2026. 1, 17
- [4] SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms, 4th edition: Priority Queues*. Disponível em: <https://algs4.cs.princeton.edu/24pq/>. Acesso em: 16 set. 2026. 1
- [5] PYTHON SOFTWARE FOUNDATION. *time — Time access and conversions: perf_counter*. Disponível em: https://docs.python.org/3/library/time.html#time.perf_counter. Acesso em: 16 set. 2026. Referência da API; não identifica a versão utilizada no experimento. 3